MİDDLEWARE

Middleware Çalışma Mekanizması

ASP.NET CORE'DA MİDDLEWARE YAPILARI

ASP.NET CORE HAZIR MİDDLEWARE'LER

Run Middleware'i

Use Middleware'i

Map Middleware'i

MapWhen Middleware'i

CUSTOM MİDDLEWARE

Custom Middleware Oluştruma

Custom Middleware Kullanimi

Middleware

Client’dan gelen request sonucunda respond dönene kadar yapılacak işlemlerin, olacak süreçlerin sorumluluğunu üstlenen ara yazılımlardır.

Kısaca request ile respons arasına giren ara yazılımlardır.

Controller’dan önce işlem yapılması, Controller’ın tetiklenme süreci, Controller’dan sonra işlem yapılması bunların hepsi birer middleware’dir.

Controller’ların ayağa kaldırılması ve tetiklenmesi de bir Middleware

Yani Request’in gelmesinden itibaren Response yapılana kadar tüm operasyonlara da middleware denilebilir.

Örneğin: Client’dan tüm kitapları döndür talebi geldi.

Önce kullanıcı doğrulanır, bir authentication yapılır. Bu işlem Request ile Response arasında yapıldığı için ayır bir Middleware’de yapılır.

Sonrasında bir Authorization yapılır kullanıcının kitaplara erişimi var mı kontrol edilir. Bu da ayrı bir Midlleware ile yapılır.

Aynı şekilde Asp.Net Core’da Controller’ların ayağa kaldırılması(initialize) da bir Middleware ile olur.

Bu yapılar her dil ve mimaride bulanabilirler Asp.Net’e özgü değiller.

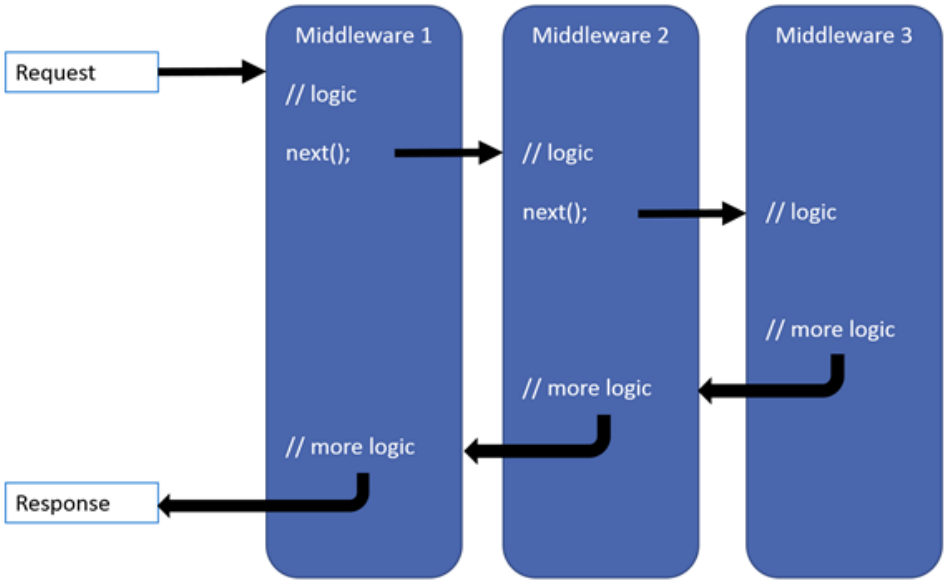
Middleware Çalışma Mekanizması

Midlleware’ler sarmal bir şekilde tetiklenirler. Yani kendisi sona ermeden başka bir Midlleware’i tetikler ve onun başlatılmasını bekler. Tetiklediği Middleware ayağa kalkmadan sonlanmaz ilgili Middleware’in ayağa kalkmasını bekler.

Mekanizma şöyle çalışıyor: İlk Midlleware tetiklenir, işini halletikten sonra kendinden sonrakini tetikler. İkinci Midlleware da işini halleder ve kendinden sonrakini tetikler ama bu süreçte 1. Ve 2. Middleware’lar sonlanmazlar. Son Midlleware da işini halleder kendinden sonra tetiklenecek Middleware olmadığı için kendinden öncekine döner ve

sonlanır. Kendinden önceki Middleware yani 2. Kendinden öncekine döner ve sonlanır. Son olarak ilk Middleware Response yapar.

Buradaki önemli kısım tüm yapı tamamlanmadan Middleware'lar sonlanmazlar.



Asp.Net Core'da Middleware yapıları

Asp.Net Core standart Framework'ün aksine Midlleware yapılanmalarını içerisinde barındırır.

Asp.Net Core'da tüm çekirdek Middleware yapıları ile çalışıyor denebilir.

Program.cs içerisindeki "Use" ile başlayan tüm yapılar(Eskiden Configure fonksiyonu içerisine yazılıryorlardı.) Middleware görevi görüyor aslında.

Request geldiği zaman Pipeline içerisindeki bu Middleware'ler tek tek tektiklenir.

Eskiden Proje ayağa kalkınca Configute metdou içerisindeki Midlleware'ler Pipeline'a ekleniyordu .Net 6 sonrasında Configure fonksiyonu kullanılmıyor.

Temel yapı, proje ayağa kalkınca Program.cs içerisinde belirtilen Middleware'ler Asp.Net Core Pipeline'ına eklenir. Sonrasında her istek geldiğinde ilgili Middleware'ler Pipeline içerisinde sırası ile çalışılar. Tabi ki Middleware mantığı ile yani biri diğerini tetikler Middleware'ler bitince en baştakine dönüp Request yapar ve sonlanırlar.

Örneğin MVC porjesi oluşturulunca hazır gelen Middleware'ler:

- app.UseAuthentication();
- app.UseAuthorization();
- app.MapControllers();

Proje ayağa kalkınca bu Middleware'ler Pipeline'a eklenir ve istek geldiği zaman şu şekilde çalışır:

Authentication → Authorization → Endpoint (Controller)

MapControllers MiddleWare'i zaten istenen Controller'ı ayağa kaldıran Middleware'dir.

Middleware'lerde tetiklenme sırası önemlidir. Biri diğerini tetiklediği için hangisinin önce geleceği önemlidir. Mesela Asp.Net Core'da önce Authentication Middleware'inden sonra Authhorization Middleware'i çağırıdır. Zaten mantıken önce kullanıcı tanınır sonra yetki kontrolü yapılır.

Asp.Net Core Hazır Middleware'ler

- Run
- Use
- Map
- MapWhen

Bunlar mimari içerisinde gelen hazır Middleware'lerdir.

Bu Middleware'ler bir görev vermeden yapmazlar. Yani arka planda çalışmazlar sadece geliştiricinin isteği halinde iş yaparlar.

Bu Middleware'ler mimarinin içerisinde gelir.

Run Middleware'i

Run Middleware'i, kendisinden sonra gelen Middleware'i tetiklemez. Sonrasında bir tetikleme olmadığı için pipeline'ı tamamlar. Yani en son kendisi olduğu için geriye doğru gider ve son olarak Response yapılır.

Kendisinden sonra bir Middleware tanımlansa bile onu tetiklemez.

Kısaca akışı keser de denebilir.

Bu etkiye Short Circuit(Kısa devre) denir.

```
app.Run(async c => { await c.Response.WriteAsync("Middleware 1"); });
app.Run(async c => { await c.Response.WriteAsync("Middleware 2"); });


app.UseHttpsRedirection();
app.UseRouting();

app.UseAuthorization();

app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();


app.Run();
```

Bu kodda ilk Run fonksiyonundan sonra gelen hiçbir Middleware tetiklenmeyecektir.

Hiçbir Controller dönmeyecektir çünkü UseRouting Middleware'ine bile ulaşamadı pipeline.

Zaten MVC projesi oluşturunca Program.cs dosyasının en sonunda app.Run() fonksiyonu bulunmaktadır.

Use Middleware’i

Normal Middleware’dir. Kendinden sonrakini çağırır, operasyon bitince kendinden öncekini tetikler, geriye response döner. Dümdüz vatandaş bir Middleware.

Async bir şekilde iki adet parametre alır. “context” ve “task”. İçerisinde parametre olarak aldığı task delegate’i ile bir Invoke() fonksiyonu kullanır. Aslında bu kendinden sonraki Middleware’lerin işlerini bitirip geri döndüğü zaman kodun nereden devam edeceğini bildirir.

Invoke metodu bir sonraki Middleware’i tetikler eğer bu fonksiyon yazılmazsa bir sonrakini tetiklemez.

Zaten UserRouting, UseAuthorization gibi fonksiyonlar da kendi içlerinde bu fonksiyona sahiptir zaten.

```
app.Use(async (context, next) =>
{
    Console.WriteLine("Start Use Middleware");
    await next.Invoke(); // bir sonraki Middlew
    Console.WriteLine("Stop Use Middleware");

});

app.Run(async c =>
{
    Console.WriteLine("Run Middleware");
});
```

Önce “Start Use Middleware” yazdırılır. Sonra Run Middleware’ına geçilir.

“Run Middleware” yazdırılır.

Run Middleware’inin işi bitince geri döner.

Invoke kısmında kaldığı yerden devam eder Use ve “Stop Use Middleware” yazdırılır.

Map Middleware'i

Bazen Middleware'i talper gönderen path'e göre filtrelemek gerekebilir. Yani şu Path'e(Route'a) bir istek geliyorsa şu Middleware'ı çalıştır gibi. Bu işlem if-else kullanarak Use yada Run Middleware'leri ile de yapılabilir. Fakat Map ile yapmak daha profesyonaldir.

```
app.Map("/Merhaba", builder =>
{
    builder.Run(async c => await c.Response.WriteAsync("Merhaba tetiklendi"));
});
```

MapWhen Middleware'i

Map Middleware'ının bir gelişmiş hali.

Map Middleware'ında request'in path'ine göre filtreleme yapılırken, MapWhen Middleware'ında Request'in belirli özelliklerine de bakılabilir.

```
app.MapWhen(c => c.Request.Method == "GET", builder =>
{
    builder.Run(async c => await c.Response.WriteAsync("Get isteği"));
});
```

Gelen Request GET ise ilgili işlem yapılıyor.

Map ve MapWhen fonksiynlarının içerisinde Run Middleware'i kullanılması zorunlu değiş Use da kullanılabilir.

Custom Middleware

Default gelen Middleware'lar dışında custom Middleware'lar da oluşturulabilir.

Bu işlem için bir sınıf oluşturulur, ardından Program.cs içerisinde extention fonksiyon olarka çağırılır.

Tüm Middleware'lar app parametresi üzerinden çağırılıyor. Bundan dolayı olarak oluşturulacak sınıf IApplicationBuilder'a dahil edilirse otomatikmen app parametresi üzerinden çağırılabilir.(Eskiden Configure fonksiyonuna verilen bir parametreydi app, şu anda configure fonksiyonu yok ama app hala var. Builder.Build() fonksiyonu ile oluşturuluyor.)

Aslınca çoğu işekm Use, Run, Map, MapWhen gibi Middleware'ler ile yapılabilir fakat amaç olabildiğince Program.cs içerisini sade tutmak.

Custom Middleware Oluştruma

Proje altında bir Middlewares klasörü oluşturulur.

Klasör içerisinde "İsim + Middleware" isimli bir class oluşturulur. Bu mecburi değil. Raconsaldır.

Mimari isimden veya yerden değil, IApplicayionBuilder'a Extantion olarak eklendiği zaman tanır Middleware'ı.

```
RequestDelegate _next;
public HelloMiddleware(RequestDelegate next)
{
    _next = next;
}
```

Bu sınıf Consturactor'una bir RequestDelegate alır parametre olarak.

Ardından bir önceki Mİddleware'in bu Middleware'i tetikleyebilmesi için bir imza fonksiyonu oluşturulur.

```
public async Task Invoke(HttpContext httpContext)
{
    // Custom Operasonyon...

    Console.WriteLine("Selamin Aleykumm"); // Midc
    await _next.Invoke(httpContext); // Bir sonrak
    Console.WriteLine("Aleykum Selam"); // Diğer M
}
```

Sınıfı oluşturduktan sonra Extention olarak eklemek için proje altında "Extentions" isimli bir klasör oluşturulur. Bu klasörün içerisine bir sınıf oluşturulur ismi çok önemli değil. Bu sınıf static olarak oluşturulur.

Sınıf içerisinde IApplicationBuilder döndüren bir fonksiyon yazılır. Bu fonksiyon parametre olarak IApplicattionBuilder alır ve içerisinde aldığı parametre ile UseMidlleware fonksiyonunu kullanır. Bu fonksiyon IApplicationBuilder içerisine ilgili Middleware Class'ını ekler. Ardından bu fonksiyon return edilir.

```
0 references
static  class Extention
{
    0 references
    public static IApplicationBuilder UseHello(this IApplicationBuilder applicationBuilder)
    {
        return applicationBuilder.UseMiddleware<HelloMiddleware>();
    }
}
```

Custom Middleware Kullanımı

Aynı normal bir Middleware kullanır gibi Program.cs içerisinde app parametresi üzerinden istenilen Middleware'e ulaşılır tabi namespace'i eklemek gerek.

```
pp.UseHttpsRedirection();
app.UseRouting();
```

```
app.UseAuthorization();

app.UseHello();

app.MapStaticAssets();
```

Eğer Custom Middleware içerisinde next.Invoke kullanılmaz ise bir sonraki Middleware’e geçmez, kısa devre olur.